Multi-Cloud IAM Anomaly Detection Platform

Technical Project Documentation

Prepared by Chinthan Dinesh

github.com/DChinthan/iam-anomaly-detector

An unsupervised machine-learning platform that detects anomalous IAM/CloudTrail user behavior and generates AI-assisted incident reports. AWS is the fully working reference implementation, end to end. GCP has ingestion, persistence, and validated IaC in place, with the compute adapter as the one remaining integration gap -- see Section 15.

Documentation snapshot: verified against application code as of commit b35ba80.

Python | scikit-learn | TensorFlow | AWS | GCP | Terraform | Kafka | Claude API

Table of Contents

1. Executive Summary

This detects anomalous AWS IAM and GCP IAM user behavior with an unsupervised ensemble of three models, trained only on normal behavioral data. Claude writes a short incident summary -- attack pattern, key signals, one-line recommendation -- for whatever gets flagged, with a rule-based fallback so the pipeline runs the same with or without an API key.

CLOUD_PROVIDER=awslgcp is read once at the CLI entry point and picks which ingestion client and persistence store handle a given run; the scoring and feature-engineering code never branches on provider. That routing covers the CLI (main.py) -- it does not currently cover the Streamlit dashboard, which is hardcoded to DynamoDB regardless of this setting. AWS is the fully working reference implementation end to end; GCP has ingestion, persistence, and validated IaC in place, with the compute adapter as the one remaining integration gap (Section 15). Section 6 goes through exactly which modules are shared and which are cloud-specific, verified by checking imports, not by reading my own comments.

What's in here

- **Detection:** 3-model unsupervised ensemble with a second confidence signal from the autoencoder's own threshold
- **Cloud portability:** CLI ingestion/storage/streaming route between AWS and GCP; the dashboard does not (see Section 6)
- **Streaming:** Event-streaming path (Kafka / GCP Pub/Sub) alongside the batch pipeline -- see Section 15 for the real accuracy/latency tradeoff this introduces
- **Incident summaries:** Claude-generated, with a response cache keyed on (user_id, score bucket) to cut redundant calls
- **Infrastructure as Code:** Two Terraform modules, one per cloud, both pass terraform validate -- neither has been applied to a real account
- **Governance:** Statistical model-drift monitoring (PSI/KS) and role-gated dashboard login

2. Problem Statement

Credential-based attacks -- stolen or leaked access keys, over-privileged accounts, insider misuse of legitimate access -- use valid credentials to operate. Because nothing about the request is malformed or unauthorized at the protocol level, signature-based and rule-based security tooling cannot detect them. The only reliable detection strategy is behavioral: does this account's current access pattern resemble how its legitimate owner normally behaves?

This is a continuously relevant problem in any organization operating cloud infrastructure -- compromised developer credentials, leaked CI/CD service-account keys, departing employees retaining access, and third-party vendor credentials being reused outside their intended scope are all instances of the same underlying failure mode. I built this as a cloud-portable pattern for that reason -- the CLI's ingestion and storage adapters can be swapped without touching the ML/scoring code (Section 6 covers exactly where that line is).

3. System Architecture

End-to-end data flow from raw event ingestion through to analyst-facing output:

Steps 2 through 7 are provider-agnostic -- the same feature-extraction, ensemble-scoring, drift-monitoring, and GenAI-reporting code executes regardless of whether events originated on AWS or GCP (verified by checking imports, see Section 6). Step 8 is the exception: the dashboard does not read CLOUD_PROVIDER and always uses DynamoDB with freshly generated synthetic data.

4. Core Components

Module	Responsibility
data/log_generator.py, lanl_adapter.py	Synthetic log generation; real-world LANL dataset adapter
data/db.py	SQLAlchemy engine abstraction - SQLite (dev) / Postgres (prod)
features/extractor.py	FeatureExtractor - computes 12 behavioral features per user
models/detector.py, autoencoder.py	3-model ensemble (Isolation Forest, One-Class SVM, TF Autoencoder), confidence tiering
models/drift.py	PSI / Kolmogorov-Smirnov statistical drift detection vs. training baseline
genai/insights.py, cache.py	Claude-powered incident report generation with response caching
aws/cloudwatch_client.py, dynamodb_store.py	AWS ingestion (CloudWatch Logs Insights) and persistence (DynamoDB)
gcp/cloud_logging_client.py, firestore_store.py	GCP ingestion (Cloud Logging) and persistence (Firestore)
streaming/event_stream.py, pubsub_backend.py, stream_processor.py	Event-bus abstraction (Kafka / Pub-Sub / mock) and incremental rescoring
dashboard/app.py, auth.py	Streamlit dashboard with role-based login. app.py hardcodes DynamoDBStore -- ignores CLOUD_PROVIDER
infra/terraform/, terraform-gcp/	Independent, validated Infrastructure-as-Code modules per cloud
main.py	CLI entry point; CLOUD_PROVIDER-aware command routing

5. ML Detection Engine

Why an Ensemble of Three Models

Each model captures a different geometric notion of "normal," so a single blind spot in one does not become a blind spot in the system:

- **Isolation Forest (40% weight):** Tree-based; isolates anomalies via random partitioning. Fast, handles high-dimensional data, assumes roughly linear separability.
- **One-Class SVM (30% weight):** RBF-kernel hypersphere around normal behavior in kernel space; captures non-linear boundaries Isolation Forest can miss.
- **TensorFlow Autoencoder (30% weight):** Dense 12-8-4-8-12 network learning to reconstruct normal sessions; high reconstruction error signals deviation the other two may not isolate cleanly.

Confidence Tiering

The autoencoder's own 95th-percentile training-reconstruction-error threshold is evaluated independently of the 0.65 ensemble cutoff. A flagged user is marked CONFIRMED when both signals agree and SUSPECTED when only the ensemble score crosses threshold -- this tells you which flags the three models actually agree on, without re-deriving it from the raw per-model scores yourself.

Model Drift Monitoring

Population Stability Index (PSI) and a Kolmogorov-Smirnov test are computed per feature against a training-time baseline snapshot, automatically saved on every model fit. This is aimed at catching silent accuracy decay -- a model that's quietly gone stale shows up as a number here instead of nothing. See Section 15 for the caveat on the hyperparameters this baseline depends on.

Behavioral Features (12 total)

Feature	Signal
total_api_calls, unique_ips/regions	Automated abuse; credential sharing
off_hours_ratio, weekend_ratio	Compromised credentials used outside normal schedule
mfa_usage_rate	Stolen long-lived access keys lacking MFA
suspicious_api_ratio	Privilege-escalation API usage (CreateAccessKey, AttachUserPolicy, etc.)
burst_score	Automated credential harvesting / scripted access
error_rate	Permission probing
geo_deviation_score	Lateral movement / IP hopping
avg/max session_duration	Persistent unauthorized sessions

6. Multi-Cloud Implementation

What's actually shared, checked by grepping every module for aws.* or gcp.* imports rather than trusting my own comments: features/extractor.py, models/detector.py, models/autoencoder.py, models/drift.py, genai/insights.py, genai/cache.py, data/db.py, and streaming/event_stream.py have zero cloud imports -- same code regardless of provider. Ingestion and persistence are separate files per cloud (aws/cloudwatch_client.py vs. gcp/cloud_logging_client.py, aws/dynamodb_store.py vs. gcp/firestore_store.py) unified only by matching method signatures, not shared code -- swapping providers means swapping which file runs, not running the same file twice.

The dashboard is the exception: dashboard/app.py imports DynamoDBStore directly and has no CLOUD_PROVIDER branch at all. Set that variable to whatever you want -- the dashboard always talks to DynamoDB and always regenerates its own synthetic dataset, never real ingested events from either cloud. The CLI's routing (below) is real; the dashboard isn't part of it yet.

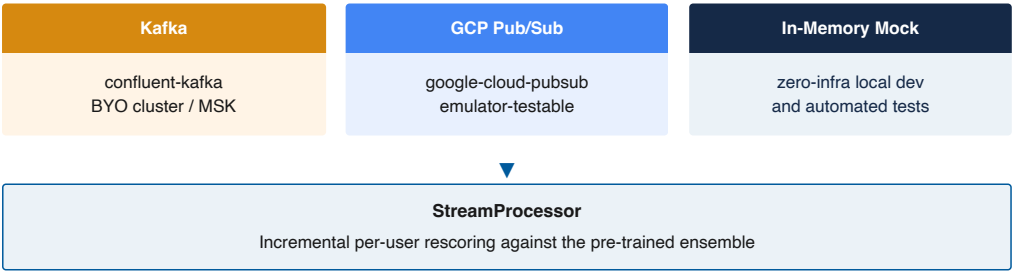
AWS	GCP
Audit log ingestion CloudWatch Logs Insights	**Audit log ingestion** Cloud Logging (Audit Logs)
NoSQL persistence DynamoDB	**NoSQL persistence** Firestore
Batch compute Lambda (container image)	**Batch compute** Cloud Run (container image)
Scheduler EventBridge	**Scheduler** Cloud Scheduler
Identity IAM role (least privilege)	**Identity** Service account (least privilege)

Cloud-Provider Routing

CLOUD_PROVIDER=awslgcp is read once at the CLI entry point and determines which ingestion client and persistence store are instantiated for that run. The streaming processor accepts its persistence backend as a constructor parameter, so the same incremental-rescoring engine works against either store without modification to its scoring logic.

7. Real-Time Streaming Architecture

A live event-streaming path runs alongside the batch pipeline: events are consumed one at a time, appended to a per-user rolling window, and any user whose window changed is rescored incrementally against the pre-trained ensemble, rather than waiting for the next full batch run.



All three backends implement the same producer/consumer interface (produce / poll / flush / close), selected at runtime via STREAM_MODE=kafka|pubsub|mock. The Pub/Sub backend is fully testable against the official local Pub/Sub emulator, requiring no live GCP project.

8. Infrastructure as Code

Two independent Terraform (HCL) modules provision the full supporting infrastructure per cloud, each validated with terraform validate.

Resource Type	AWS Module	GCP Module
Result storage	DynamoDB table + GSI	Firestore (Native mode) database
Log aggregation	Per-region CloudWatch log groups	Cloud Logging sink + bucket
Messaging	(Kafka - BYO cluster)	Pub/Sub topic + subscription
Scheduled compute	Lambda (container image)	Cloud Run (container image)
Scheduler	EventBridge rule	Cloud Scheduler job
Identity	Least-privilege IAM role	Least-privilege service account
API enablement	Not required (enabled by default)	google_project_service - 8 APIs
Optional auth	Cognito user pool	-
Monitoring	CloudWatch metric alarm on errors	-

Publishing the scoring service's container image is outside either module's ownership -- Terraform provisions the compute target, not the image. On the GCP side this is more than a packaging step: the only application code that exists (infra/lambda/scorer/handler.py) is a Lambda-style handler(event, context) function that assumes /var/task. It would fail Cloud Run's health check, since Cloud Run needs a container that binds an HTTP server to $PORT. Writing that adapter is real, not-yet-done work -- see Section 15.

9. Estimated Infrastructure Cost

ESTIMATED (Terraform-defined resources, not deployed)

Neither Terraform module has been applied to a real account (Section 8, Section 15), so nothing on this page is a measured bill. Every figure below is a unit-price calculation against the resources each module actually defines, run against the same 55-user / ~85,000-event, 30-day pattern used for the detection-rate figure in Section 12, plus a 10x projection.

Method

Manual calculation against current AWS/GCP public pricing pages -- not Infracost. Infracost was installed and run against both modules, but `infracost breakdown` (now aliased to `infracost scan`) requires an interactive OAuth login to Infracost's hosted pricing API before it prices anything; that login wasn't pursued for a documentation estimate, so no Infracost output exists here. Pricing pulled 2026-07-13 from the AWS Lambda, DynamoDB on-demand, and CloudWatch pricing pages, and the GCP Cloud Run, Firestore, Pub/Sub, Cloud Scheduler, and Cloud Logging pricing pages. Every number below is (usage assumption) × (public list price) -- reproducible from those two inputs alone.

Usage assumptions (flagged -- not measured, no deployed instance to profile): scoring runs on each module's default schedule (`rate(15 minutes)` / `*/15 * * * *`) = 2,880 runs/month, one result write per user per run; reads assumed at 10% of writes (no real read pattern -- the dashboard is synthetic-only per Section 6); average Lambda/Cloud Run duration assumed at 10s at tested scale, 40s at 10x (sub-linear growth from fixed overhead).

Resource Type	AWS Module — Estimated	GCP Module — Estimated
Result storage	$0.40/mo — DynamoDB on-demand, ~316,800 WRU/mo (table + GSI) × $1.25/M WRU. No free tier applies to on-demand mode.	$0.00/mo net — Firestore, ~158,400 writes/mo (~5,280/day), under the 20,000 writes/day perpetual free quota.
Log aggregation	$0.00/mo net — ~25.5MB/mo ingested, under CloudWatch Logs' 5GB/mo free tier.	$0.00/mo net — ~25.5MB/mo ingested, under Cloud Logging's 50GiB/mo free tier.
Messaging	Not calculable — (Kafka - BYO cluster); this module provisions no broker (Section 8).	$0.00/mo net — Pub/Sub, ~25.5MB/mo throughput, under the 10GiB/mo free tier.
Scheduled compute	$0.00/mo net — Lambda, 2,880 invocations/mo × ~10s × 1024MB, under the 400,000 GB-s/mo free tier.	$0.00/mo net — Cloud Run, same cadence, 1 vCPU / 1GiB, under the 180,000 vCPU-s / 360,000 GiB-s/mo free tier.
Scheduler	$0.00/mo — default-bus EventBridge rules aren't billed separately from the target invocation.	$0.00/mo — 1 job, under the 3 free jobs/mo (billing-account level).
Identity	$0.00/mo — IAM roles carry no charge.	$0.00/mo — service accounts carry no charge.
API enablement	n/a	$0.00/mo — enabling APIs has no charge; usage is priced in the rows above.
Optional auth	$0.00/mo — not deployed (enable_cognito_auth = false by default).	—
Monitoring	$0.10/mo — 1 standard-resolution CloudWatch alarm (scorer_errors).	—

9. Estimated Infrastructure Cost

ESTIMATED (Terraform-defined resources, not deployed) — continued

Total Estimated Monthly Cost — tested scale (55 users, ~85,000 events/month)

AWS: $0.99/month at current list prices; **$0.50/month** net of AWS's standard free-tier allowances (Lambda's 400,000 GB-seconds + 1,000,000 requests) — DynamoDB on-demand and CloudWatch Alarms have no equivalent free allowance, which is why AWS doesn't net to zero the way GCP does below.
GCP: $1.06/month at current list prices; **$0.00/month** net of GCP's standard free-tier allowances — Cloud Run's 180,000 vCPU-s/360,000 GiB-s, Firestore's 20,000 writes + 50,000 reads/day, Pub/Sub's 10GiB/month, and Cloud Logging's 50GiB/month all cover this workload outright at this volume.

Projected at 10x scale (550 users, ~850,000 events/month, same 15-minute cadence)

AWS: $6.18/month gross, **$4.14/month** net — DynamoDB on-demand has no free-tier offset at any scale, so its cost (now $4.04/month) carries through regardless; Lambda and the alarm stay inside or near their free allowances.
GCP: $6.13/month gross, **$1.77/month** net — Firestore's daily write quota (20,000/day) is the only allowance this projection exceeds (~52,800 writes/day at 10x); Cloud Run, Pub/Sub, and Cloud Logging all stay inside theirs.

Cost per 1,000 events (infrastructure only)

Excludes Claude API usage, which Anthropic bills separately and which isn't a Terraform-defined resource.
AWS: $0.012/1,000 events at tested scale, $0.007/1,000 events at 10x.
GCP: $0.012/1,000 events at tested scale, $0.007/1,000 events at 10x.
Both drop at higher scale for the same reason: the scoring compute (Lambda / Cloud Run) runs on a fixed 15-minute schedule (`var.schedule_expression`), not per event, so its cost doesn't scale linearly with event volume the way the per-write storage cost does.

> Not priced: Kafka. Section 8 already lists AWS messaging as "(Kafka - BYO cluster)" -- Terraform provisions no broker here, so there's no Terraform-defined resource to attach a number to. Actual cost depends entirely on whatever cluster (self-hosted, MSK, Confluent Cloud) it's pointed at.

10. Security, Access Control & Testing

Least-Privilege Access

Both cloud IAM identities are scoped to only the permissions their workload requires -- no wildcard actions, no project-owner or editor roles. Secrets (the Claude API key) are never hardcoded, routed instead through environment variables locally and Secrets Manager / Secret Manager in each Terraform module.

Dashboard Role-Based Access Control

The dashboard enforces its own three-tier login -- admin, analyst, and viewer -- gating retrain controls, GenAI alerts, and user-identifying data by role. This is a DASHBOARD_USERS env var, SHA-256 hashed and checked in-process -- not backed by a real identity provider. Fine for local use, not something to point real users at without replacing it (Cognito is scaffolded in infra/terraform/cognito.tf but nothing wires the dashboard to it yet).

Automated Test Coverage

Test Suite	Tests	Covers
test_feature_extractor.py	15	Per-feature correctness across all 12 behavioral signals
test_detector.py	6	Ensemble fit / score / save / load, confidence tiering
test_log_generator.py	6	Schema integrity, anomaly injection, timestamp validity
test_dynamodb_store.py	5	AWS persistence layer, mock-mode CRUD
test_firestore_store.py	5	GCP persistence layer, mock-mode CRUD
test_pubsub_backend.py	3	Streaming round-trip, lazy-import safety

Total: 40 tests, 100% passing, zero cloud credentials required to execute any of them.

11. Technology Stack

Layer	Technologies
ML / Modeling	scikit-learn (Isolation Forest, One-Class SVM), TensorFlow / Keras
GenAI	Anthropic Claude API
Feature Engineering	pandas, numpy, scipy
Relational Storage	SQLite (dev), PostgreSQL - AWS RDS/Aurora or GCP Cloud SQL (prod), via SQLAlchemy
NoSQL Storage	AWS DynamoDB, GCP Firestore
Event Streaming	Apache Kafka (confluent-kafka), GCP Pub/Sub
Infrastructure as Code	Terraform (AWS + Google providers), Docker
Dashboard	Streamlit, Plotly
Testing	pytest (40 tests)
Version Control	Git / GitHub

Command-Line Interface

Representative commands from the CLI entry point (main.py):

```
python main.py pipeline         # generate -> train -> score -> insights
python main.py ingest            # CLOUD_PROVIDER-aware ingestion
python main.py stream-demo 200 # end-to-end streaming demo, zero infra
python main.py drift             # model drift report vs. training baseline
streamlit run dashboard/app.py # launch the analyst dashboard
```

12. Repository & Project Information

Field	Value
Repository	github.com/DChinthan/iam-anomaly-detector
License	MIT (open source)
Primary language	Python 3.11+
Automated tests	40 tests, 100% passing
Infrastructure validation	terraform validate: Success (AWS + GCP modules)
Detection rate (proof of concept)	100% true positive / 0% false positive on the most recent clean run -- see caveat below

That detection rate is one run against synthetic data with injected, labeled anomaly patterns, not a validated benchmark -- injected anomalies are cleaner and more separable than real attacker behavior that's actively trying to blend in. Reproduce it with `rm -f data/iam_logs.db && python main.py pipeline`; expect the exact event count and rate to vary slightly since the log generator isn't seeded. The LANL real-world adapter (Section 3) proves schema compatibility with real auth data, not detection accuracy -- it doesn't ingest LANL's ground-truth redteam labels, so there's no real-world precision/recall number yet. This documentation reflects the platform's architecture and these verified, reproducible figures as of the commit noted on the cover page -- not a claim of production validation.

13. Verification Evidence

Actual terminal output from the commands cited throughout this document, captured directly from the local development environment immediately prior to publication.

Automated Test Suite

```
$ python -m pytest tests/ -v
collected 40 items

tests/test_detector.py ......................... 6 passed
tests/test_dynamodb_store.py ..................... 5 passed
tests/test_feature_extractor.py .................. 15 passed
tests/test_firestore_store.py ................... 5 passed
tests/test_log_generator.py ..................... 6 passed
tests/test_pubsub_backend.py ..................... 3 passed

====================== 40 passed in 24.41s ======================
```

Infrastructure Validation -- AWS Module

```
$ cd infra/terraform && terraform init -backend=false && terraform validate
Terraform has been successfully initialized!

Success! The configuration is valid.
```

Infrastructure Validation -- GCP Module

```
$ cd infra/terraform-gcp && terraform init -backend=false && terraform validate
Terraform has been successfully initialized!

Success! The configuration is valid.
```

Git State

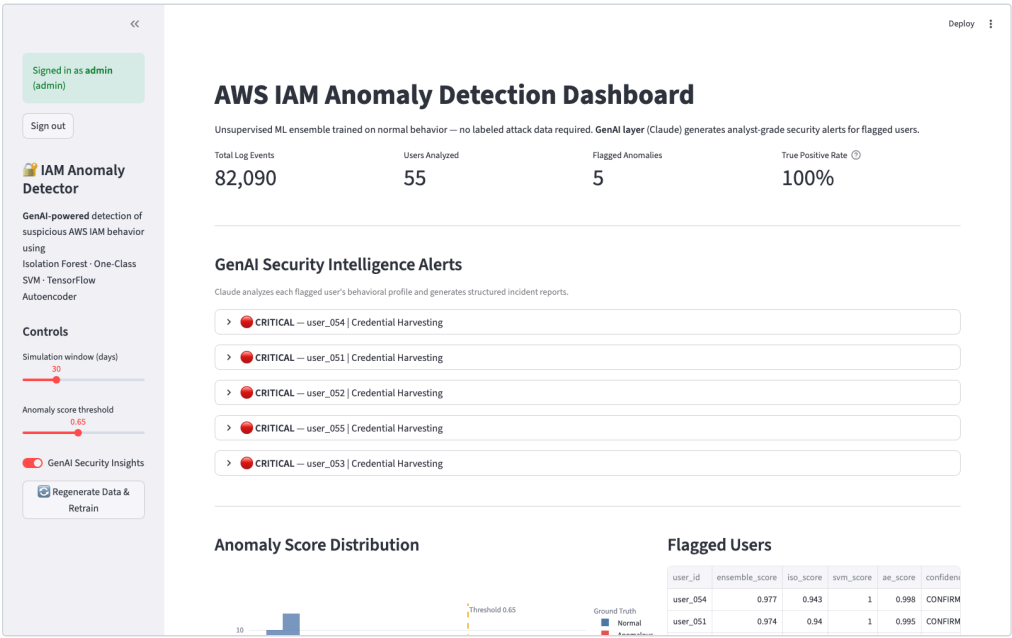
```
$ git status --short
(clean -- no output)
$ git rev-parse HEAD origin/main
b35ba80...  (HEAD)
b35ba80...  (origin/main -- identical, fully pushed)
```

14. Live Dashboard

Screenshots of the Streamlit dashboard running locally against a freshly generated synthetic dataset (55 users, ~85,000 events), authenticated as the admin role via the dashboard's own role-based access control. Not a mockup -- captured directly from the running application.

Overview: Metrics and GenAI Security Alerts

All 5 synthetically injected anomalous users are correctly flagged as CRITICAL / Credential Harvesting, with a 100% true positive rate on this run (0 false positives across the 50 normal users) -- consistent with the proof-of-concept figure in Section 12.

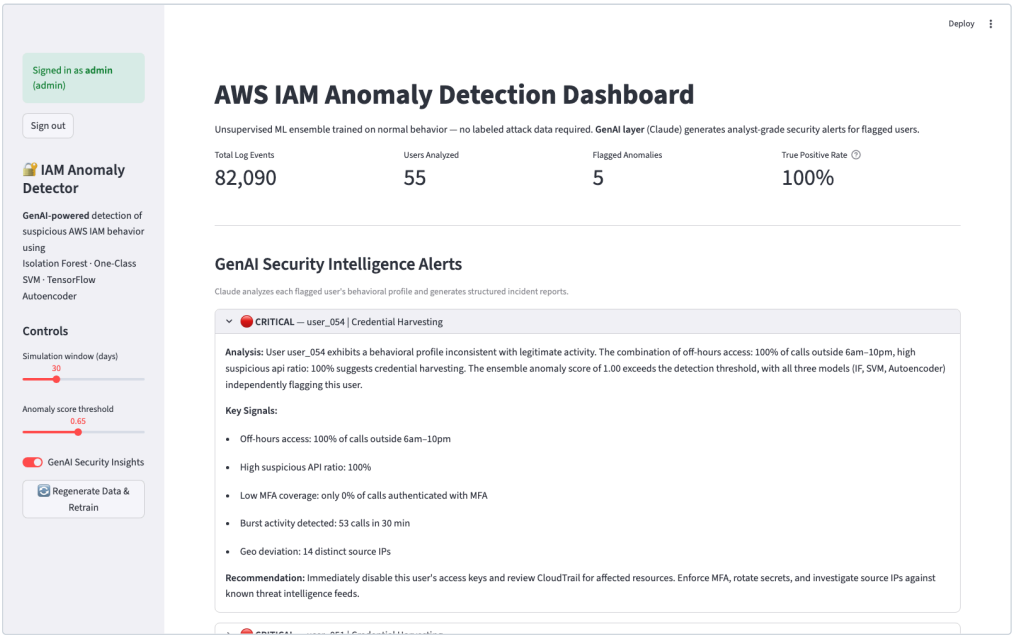


14. Live Dashboard

continued

GenAI Incident Report Detail

An expanded incident report for a flagged user, showing the specific behavioral signals driving the alert (off-hours access, suspicious API ratio, MFA coverage, burst activity, geographic deviation) and the generated remediation recommendation.

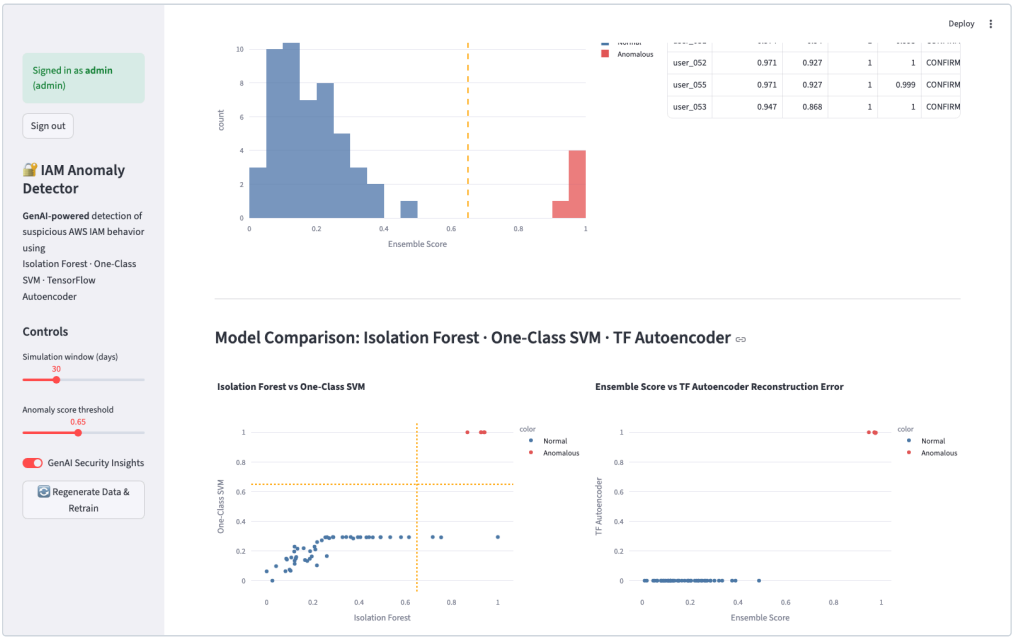


14. Live Dashboard

continued

Score Distribution and Model Comparison

The ensemble score histogram shows a clean separation between normal and anomalous users at the 0.65 threshold. The pairwise model-comparison scatter plots show all three models -- Isolation Forest, One-Class SVM, and the TensorFlow Autoencoder -- independently agreeing on the same five anomalous users, which is what the confidence-tiering mechanism relies on.



15. Limitations / Known Gaps / What I'd Improve Next

Specific things found by checking the code, not a generic disclaimer paragraph:

- **Dashboard isn't cloud-portable:** dashboard/app.py hardcodes DynamoDBStore and ignores CLOUD_PROVIDER -- it always writes to DynamoDB and always uses freshly generated synthetic data, never real ingested events from either cloud.
- **Dashboard auth isn't a real identity provider:** DASHBOARD_USERS env var, SHA-256 hashed, checked in-process. Cognito is scaffolded in Terraform but not wired to the dashboard.
- **GCP Cloud Run scoring service has no working entrypoint:** The only application code that exists is a Lambda-style handler(event, context) function assuming /var/task -- it would fail Cloud Run's health check, which needs an HTTP server on $PORT. That adapter isn't written yet.
- **DynamoDB GSI type mismatch:** infra/terraform/dynamodb.tf types the flagged GSI key as a String; aws/dynamodb_store.py writes it as a native Python bool via boto3. The index wouldn't actually match real writes without changing one side or the other.
- **terraform validate is not terraform apply:** Both modules validate; neither has been applied to a real AWS or GCP account. Quota limits, region availability, and IAM propagation delays only show up on a real apply.
- **Section 9's cost figures are estimates, not measured billing:** they're unit-price calculations against the resources each Terraform module defines -- the same "validate, not apply" gap above means there's no real invoice to check them against. Real usage patterns (retry storms, larger payloads, actual read/write ratios) would move the numbers; the assumptions behind them are stated in Section 9 so they can be re-run if that ever changes.
- **DynamoDB on-demand has no cost ceiling at scale, and hasn't been re-evaluated against provisioned capacity:** variables.tf defaults dynamodb_billing_mode to PAY_PER_REQUEST, which is the right call at demo/tested volume (Section 9) since it avoids paying for idle provisioned throughput. But on-demand carries a per-request premium with no equivalent to Firestore's always-free daily quota, and that premium compounds linearly with no ceiling as traffic grows. If this ran at sustained, predictable high volume, PROVISIONED capacity with auto-scaling would likely be cheaper -- untested, since there's no deployed traffic to benchmark it against.
- **Lambda and Cloud Run are sized identically without documented benchmarking:** infra/terraform/lambda.tf sets memory_size = 1024 and infra/terraform-gcp/cloud_run.tf sets memory = "1Gi" / cpu = "1" -- the same allocation on both clouds, chosen without a profiling run against the actual scoring workload's peak memory/CPU use. If real usage is materially lower, both are paying for allocated compute the workload doesn't use; if it's higher, both could be under-provisioned. Neither has been checked.
- **No real-world validated detection rate:** The only benchmark is the synthetic proof-of-concept in Section 12. The LANL adapter proves schema compatibility, not detection accuracy -- its ground-truth redteam labels aren't wired in.
- **Hyperparameters are defaults, not tuned:** Ensemble weights, the 0.65 threshold, Isolation Forest's n_estimators/contamination, the autoencoder's latent_dim/epochs -- none came from cross-validation or a grid search against labeled data.
- **GenAI caching reduces calls, doesn't cap spend:** genai/cache.py skips a repeat Claude call for an unchanged score, but there's no hard rate limit or budget ceiling on top of that.
- **suspicious_api_ratio is AWS-shaped:** The suspicious-API list is fixed AWS action names. On GCP-sourced data this feature reads near zero even for genuinely suspicious activity, since GCP method names look completely different. The other 11 features are schema-based and unaffected.
- **Streaming trades accuracy for latency:** Incremental rescoring uses small 20-event windows that produce noisier features -- especially burst score -- than the 30-day batch baseline, so the streaming path over-flags relative to batch. A real tradeoff, not a bug to fix for free.
- **No CI pipeline in this repo:** Tests and terraform validate are run manually. There's no GitHub Actions workflow gating a PR on either.